

CodeMorph AI

AI-Powered Code Translation, Explanation, Optimization & Bug Detection

Project Documentation

Charan Teja Epuri

Author

Charan Teja EPURI

Aspiring React Developer

Table of Contents

1. Introduction
2. Project Overview
3. Objectives
4. Features
5. System Architecture
6. Technologies Used
7. Project Structure
8. Project Workflow
9. Implementation Details
10. Learning Outcomes
11. Conclusion
12. Project Links & Resources
13. Author

1. Introduction

CodeMorph AI is an AI-powered developer platform designed to help developers transform, understand, improve, and debug source code using Google Gemini AI. The application combines a modern React interface with dedicated AI-powered development tools.

The project focuses on practical use of Generative AI in software development. Instead of providing unrelated utilities, CodeMorph AI concentrates on four core capabilities: code conversion, code explanation, code optimization, and bug detection.

2. Project Overview

CodeMorph AI is a single-page React application built with Vite. It provides a developer-focused workspace where users can select programming languages, enter source code, submit AI requests, and view structured results.

The application uses a reusable component architecture, React Router for navigation, Context API for theme management, Local Storage for theme persistence, and the Google Gemini API for AI-powered processing.

Core Modules

- AI Code Converter — converts code between supported programming languages.
- AI Code Explainer — provides structured explanations from beginner to advanced level.
- AI Code Optimizer — suggests optimized code and improvement areas.
- AI Bug Detector — identifies potential issues and provides suggested fixes and corrected code.
- About / Contact — presents project information, developer details, portfolio, GitHub, and LinkedIn resources.
- Settings — provides application appearance and theme controls.

3. Objectives

- Build a practical AI-integrated developer productivity application.
- Use Gemini AI to perform common source-code analysis and transformation tasks.
- Provide a clean and responsive interface for developers.
- Implement reusable React components and maintainable project structure.
- Handle AI loading, errors, retries, and response parsing reliably.

- Support multiple programming languages in the code conversion workflow.
- Present AI results in structured and understandable formats.
- Prepare the application for production deployment.

4. Features

4.1 AI Code Converter

- Source and target language selection.
- Monaco-based code editing workspace.
- Gemini-powered code conversion.
- Converted-code output.
- Copy and download actions.
- Clear and language/code swap actions.
- Code statistics and source/output comparison.
- Validation, loading, and error handling.

4.2 AI Code Explainer

- Code explanation using Gemini AI.
- Overall Summary.
- Line-by-Line Explanation.
- Program Flow.
- Beginner Explanation.
- Advanced Explanation.
- Markdown rendering.
- Structured explanation tabs.
- Copy/download support and error handling.

4.3 AI Code Optimizer

- AI-generated optimized code.
- Performance Improvements.
- Readability Improvements.
- Maintainability Improvements.
- Best Practices.
- Potential Issues.
- Original versus optimized comparison.
- Optimization statistics.
- Copy/download support and error handling.

4.4 AI Bug Detector

- Syntax and logic issue detection.
- Runtime, type, control-flow, edge-case, security, and performance checks.
- Critical, High, Medium, and Low severity classification.
- Bug summary and detailed issue cards.
- Line references, descriptions, and suggested fixes.
- Corrected code output.
- Structured JSON response parsing.
- Loading and error handling.

4.5 Application UI

- Responsive navbar and sidebar navigation.
- Dark and light themes.
- Responsive developer-focused layouts.
- Reusable buttons, cards, editors, status bars, and loaders.
- Toast notifications.
- About and Contact information.

5. System Architecture

CodeMorph AI follows a modular frontend architecture. User interactions are handled by React pages and reusable components. AI operations are centralized through the Gemini service and custom hook, while specialized parsers transform AI responses into structures suitable for the UI.

Architecture Flow

User Interface → React Page → useGemini Hook → Prompt Builder → Gemini Service → Gemini API → AI Response Parser → Structured Result → UI

Major Layers

- Presentation Layer — pages and reusable UI components.
- Routing Layer — React Router routes and layout.
- State Layer — React state and Theme Context.
- Service Layer — Gemini API communication, prompts, and retry handling.
- Utility Layer — validation, parsing, clipboard, download, and error utilities.
- Configuration Layer — Gemini model and generation settings.
- Data Layer — supported language and navigation data.

6. Technologies Used

- React 19 — frontend application development.
- Vite — development server and production build tooling.
- JavaScript (ES6+) — application logic.
- React Router DOM — client-side routing.
- Context API — application theme state.
- CSS3 — responsive styling and UI design.
- Google Gemini AI — generative AI capabilities.
- @google/generative-ai — Gemini API integration.
- React Icons — interface icons.
- React Hot Toast — user notifications.
- Monaco Editor — code editing experience.
- Git & GitHub — source control and repository management.
- Vercel — planned production deployment.

7. Project Structure

The project is organized into feature-specific pages and components with shared services, utilities, configuration, data, and styling.

codemorph-ai/

```
|—— src/
|  |—— assets/
|  |—— components/
|  |  |—— about/
|  |  |—— bug-detector/
|  |  |—— common/
|  |  |—— converter/
|  |  |—— editor/
|  |  |—— explain/
|  |  |—— layout/
|  |  |—— optimizer/
|  |  |—— settings/
|  |—— config/
|  |—— context/
|  |—— data/
|  |—— hooks/
```

```
| | | pages/
| | | | | About/
| | | | | BugDetector/
| | | | | Converter/
| | | | | Explain/
| | | | | Home/
| | | | | NotFound/
| | | | | Optimizer/
| | | | | Settings/
| | | routes/
| | | services/
| | | styles/
| | | utils/
| | | App.jsx
| | | index.css
| | | main.jsx
| | .env.example
| | .gitignore
| | package.json
| | package-lock.json
| | vite.config.js
| | README.md
```

8. Project Workflow

8.1 Code Conversion Workflow

The user selects a source language and target language, enters source code, and starts conversion. The application validates the input, builds a conversion prompt, sends it to Gemini through the service layer, parses the returned code, and displays the result. Copy, download, clear, swap, statistics, and comparison actions are then available.

8.2 Code Explanation Workflow

The user selects a programming language and enters code. The application builds an explanation prompt and sends it to Gemini. The returned response is parsed into five expected explanation sections and displayed through structured tabs with Markdown rendering.

8.3 Code Optimization Workflow

The source code and selected language are sent to Gemini with an optimization prompt. The structured response is displayed through optimization sections, together with comparison and statistics information.

8.4 Bug Detection Workflow

The selected language and source code are submitted to Gemini using the bug detection prompt. The returned JSON is validated and normalized by the bug response parser. The UI displays the summary, detected issues, severity levels, and corrected code.

9. Implementation Details

9.1 Gemini Service

The Gemini service initializes the Google Generative AI client using the API key supplied through the Vite environment variable `VITE_GEMINI_API_KEY`. It applies the configured model, generation settings, safety settings, retry behavior, and response parsing.

9.2 Prompt Service

The prompt service contains dedicated prompt builders for code conversion, code explanation, code optimization, and bug detection. Keeping prompts separate from UI components improves maintainability and keeps AI instructions consistent.

9.3 Retry Handling

The retry service identifies transient errors such as network failures, timeouts, rate limits, and selected server errors. It retries eligible requests using exponential backoff.

9.4 Response Parsing

Specialized parsers process AI output before it reaches the interface. The explanation parser identifies the five required explanation sections, while the bug parser validates JSON, normalizes bug records, calculates severity summaries, and extracts corrected code.

9.5 Error Handling

AI errors are converted into user-friendly messages for API key problems, network failures, timeouts, rate limits, service unavailability, empty responses, and unexpected failures. Detailed error logging is limited to development environments.

9.6 Environment Configuration

The Gemini API key is read from an environment variable rather than being hard-coded. The .env file is excluded from version control, while .env.example documents the required variable without exposing credentials.

9.7 Production Build

The application was validated using the Vite production build command. The production build completed successfully during the production-preparation phase.

10. Learning Outcomes

- Practical experience building a React application with Vite.
- Understanding of reusable component-based architecture.
- Experience with React Router and application layouts.
- Experience using Context API for global theme state.
- Integration of a Generative AI API into a frontend application.
- Prompt construction for code transformation and analysis.
- AI response parsing and validation.
- Error handling and retry strategies for API requests.
- Responsive UI development using CSS3.
- Code editor integration and developer-focused interface design.
- Environment-variable and API-key security practices.
- Production build preparation and deployment readiness.
- Git and GitHub-based incremental project development.

11. Conclusion

CodeMorph AI demonstrates how Generative AI can be integrated into a practical developer-focused web application. By concentrating on code conversion, explanation, optimization, and bug detection, the project provides a focused set of capabilities for common development workflows.

The project also demonstrates modern frontend engineering practices including reusable React components, modular services, structured response parsing, error handling, responsive design, environment-based configuration, and production build preparation.

12. Project Links & Resources

- ✓ [GitHub Repository](#)
- ✓ [Live Demo](#)

13. Author

Charan Teja EPURI

Aspiring Python Full Stack Developer

Developer with a focus on full-stack development, AI-integrated applications, and practical software projects.

Portfolio & Professional Links

- ❖ **Django Portfolio:** <https://portfolio-site-django.onrender.com>
- ❖ **GitHub:** <https://github.com/charanepuri>
- ❖ **React Portfolio:** <https://charan-react-portfolio.vercel.app>
- ❖ **LinkedIn:** <https://www.linkedin.com/in/charan-teja-972aa9231>
- ❖ **Flask Portfolio:** <https://flask-developer-dashboard-portfolio.onrender.com/>